Knowledge-Based Systems 339 (2026) 115538

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Balanced poisoning: A stealthy backdoor attack for node classification in imbalanced graph data

Haibin Tong [a], Huifang Ma [a,*], Hui Shen [a], Zhixin Li [b], Liang Chang [c]

[a] College of Artificial Intelligence and Computer Science, Northwest Normal University, Lanzhou, Gansu, 730070, China
[b] Key Lab of Education Blockchain and Intelligent Technology, Ministry of Education, Guangxi Normal University, Guilin, Guangxi, 541004, China
[c] School of Computer Science and Information Security, Guilin University of Electronic Technology, Guilin, Guangxi, 541004, China

ARTICLE INFO

Keywords:
Backdoor attacks
Graph neural networks

ABSTRACT

Graph Neural Networks (GNNs) have emerged as a forefront technique in recent graph encoding methodologies and have achieved considerable success in a variety of graph mining tasks. Research has revealed that GNNs are susceptible to backdoor attacks during node classification tasks. However, the effectiveness of these attacks is predicated on the assumption that the class distribution of the poisoned training data is balanced, an assumption that often does not hold true in real-world scenarios. In settings with class imbalance, the Attack Success Rates (ASRs) exhibit substantial variability across different classes. Moreover, crafting stealthy triggers presents a significant challenge. To tackle these issues, we introduce a novel backdoor attack method for imbalanced node classification, termed BPBA, which is designed to execute balanced and stealthy attacks. We develop a new strategy for selecting poisoned nodes and a mechanism for constructing stealthy triggers. Our strategy meticulously selects poisoned nodes from every class, ensuring accuracy and fairness, while our trigger construction approach adaptively creates triggers that are subtle, thereby enhancing resilience against defensive measures. Comprehensive experiments conducted on multi-scale datasets validate the efficacy, balance, and stealth of BPBA in the context of imbalanced data for node classification tasks.

1. Introduction

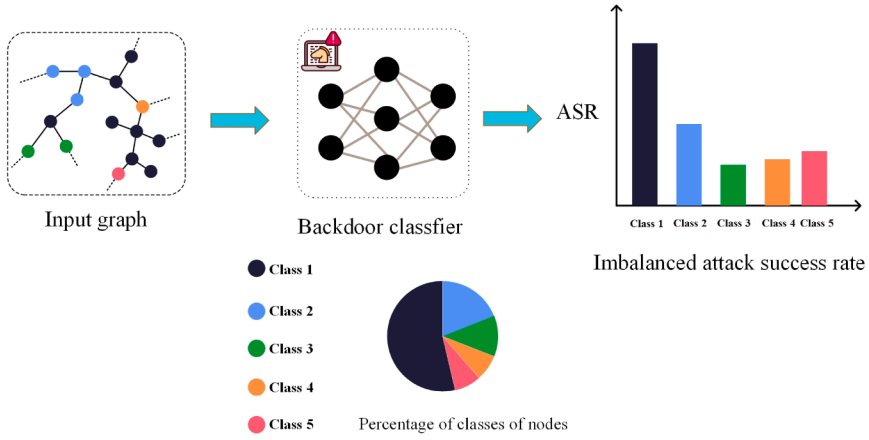
Graphs are widely used to represent complex relationships between entities in various domains, such as social networks [1,2], citation networks [3,4], and knowledge graphs [5–7]. In the field of modeling graph-structured data, GNNs have emerged as a popular and effective approach due to their remarkable efficacy and scalability [8–10]. While GNNs have achieved significant success in multiple domains, studies have revealed their vulnerability to attacks. GNNs typically employ the message passing mechanism, which makes them susceptible to graph injection attack (GIA) where attackers disrupt the original graph structure or manipulate node features. For instance, AGIA [11] selects original nodes and connects them with injected nodes using a strategy based on topological defects. They also devise an optimization objective to generate smooth features for injected nodes, thereby reducing the accuracy of GNNs in node classification tasks. However, many existing GIA methods lack sufficient stealthiness and can be easily countered by common defense techniques based on edge pruning, thus failing to provide robust protection against sophisticated attacks. While edge pruning remains an effective defense against certain types of GIA, recent advanced methods (e.g. HAO [11], CANA [12]) have significantly improved stealthiness to evade such detection techniques.

Backdoor attacks offer a promising approach to address the aforementioned limitations. These attacks involve the construction of a trigger by an attacker, which is then injected into the training dataset to create a backdoored dataset. Classifiers trained on this manipulated dataset can be controlled by the attacker. When the input data contains the trigger, the classifier produces a predetermined classification result defined by the attacker. Backdoor attacks are more stealthy compared to GIA, as they allow the creation of triggers that resemble features of poisoned nodes. Moreover, they require a smaller attack budget, reducing the risk of detection and defense. These attacks can be employed to target recommendation systems [13], node classification systems [14], and other applications. As a result, the attacker can influence the recommendation of products that users may not prefer, thereby degrading the performance of the recommendation system or causing the classifier to produce incorrect results.

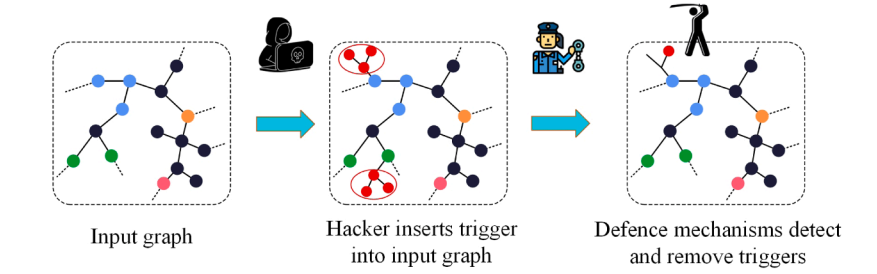
Node-level backdoor attacks become a focal point of research in recent years, with representative work by Dai et al. Dai et al. [15] designing specific strategies for selecting poisoned nodes and sample-specific

* Corresponding author.
E-mail addresses: 2022222195@nwnu.edu.cn (H. Tong), mahuifang@nwnu.edu.cn (H. Ma), 2023212026@nwnu.edu.cn (H. Shen), lizx@gxnu.edu.cn (Z. Li), changl@guet.edu.cn (L. Chang).

https://doi.org/10.1016/j.knosys.2026.115538
Received 7 April 2025; Received in revised form 1 February 2026; Accepted 8 February 2026
Available online 13 February 2026
0950-7051/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) Imbalanced ASRs caused by an imbalanced dataset.



(b) Simple trigger generation methods are easily blocked by defense mechanisms.

Fig. 1. Node-level Backdoor Attacks under imbalanced datasets. The upper part illustrates the significant variation in Attack Success Rate (ASR) between different classes when training datasets are imbalanced. The lower part demonstrates that defense methods can easily detect and remove simple trigger construction and insertion techniques, rendering them ineffective.

triggers. However, the success of these methods hinges on a significant assumption: **the class distribution of the training data for poisoned nodes is balanced**, which often does not align with reality. Class imbalance is common in the real world. For example, there are far fewer "celebrity" nodes than ordinary users in social networks, and the number of highly influential papers in citation networks is far less than that of ordinary papers. However, most existing graph backdoor attack methods implicitly or explicitly assume class balance in their design and evaluation. Class imbalance also affects the effectiveness of attacks. For example, directly applying existing methods to class-imbalanced data can lead to large and unpredictable fluctuations in attack success rates across different classes. In the context of class imbalance, there are substantial differences in ASR across different classes, as illustrated in Fig. 1(a) (detailed experimental analysis on real-world datasets are illustrated in Section 2.3). On the other hand, **designing a stealthy trigger is also a challenging task**. A stealthy trigger not only maintains the stealthiness of the attack but also enhances the robustness of the backdoor attack against defense mechanisms, as depicted in Fig. 1(b). Nevertheless, research on backdoor attacks for node classification tasks under imbalanced data settings remains relatively under-explored.

To address the aforementioned limitations, a novel backdoor attack method, termed BPBA (Balanced Poisoning Backdoor Attack), has been designed under the imbalanced data distributions scenario. Specifically, in order to address the issue of imbalanced attack effectiveness, **a dedicated strategy for selecting poisoning nodes is developed.** This strategy ensures the accurate and reasonable selection of a balanced number of poisoned nodes from each class, thereby achieving balanced ASR across different classes. To generate more stealthy triggers, **a novel trigger builder is designed** to enhance the attack's robustness. In summary, the contributions of this paper can be summarized as follows:

- We experimentally demonstrate that existing backdoor attack for node classification tasks fail to achieve balanced and effective attack results across all classes.
- We present a backdoor attack method called BPBA, which incorporates a more intelligent strategy for selecting poisoned nodes and a trigger builder capable of adaptively designing stealthier triggers.
- We extensively conduct experiments on multi-scale and imbalanced datasets. The results demonstrate the effectiveness, balance, and stealth of BPBA for backdoor attacks on node classification tasks.

The rest of the paper is organized as follows: Section 2 introduces relevant notation, the threat model, and preliminary analysis. Section 3 offers a detailed description of BPBA. Section 4 outlines our experimental setup and results. Section 5 provides an overview of related work. Finally, Section 6 concludes the paper.

2. Preliminaries

In this section, we provide a concise introduction to the symbols used in the modeling section of the article, the threat model and the preliminary analysis.

2.1. Notations and problem definition

Notations. In this work, we use $G = (V, E, \mathbf{X}, \mathbf{A})$ to represent the input undirected graph, where $V = V^L \cup V^U$ denotes the set of nodes, V^L represents labeled nodes, V^U represents unlabeled nodes, E denotes the set of edges, and $\mathbf{X} \in \mathbb{R}^{|V| \times d}$ denotes the attribute matrix of the nodes. $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$ denotes the adjacency matrix of graph G, where $\mathbf{A}_{ij} = 1$ indicates that there is an edge between nodes v_i and v_j. In addition to the

Table 1
Notations and definitions.

Notations	Definitions
$G = (V, E, \mathbf{X}, \mathbf{A})$	Graph G with node set V, V^L represents labeled nodes, V^U represents unlabeled nodes, and $V^L \cup V^U = V$; edge set E; nodes attribute matrix $\mathbf{X} \in \mathbb{R}^{\lvert V\rvert \times d}$; $\mathbf{A}$ is adjacency matrix of G, where $\mathbf{A}_{ij} = 1$ if v_i and v_j are connected
$f_\theta(\cdot)$	GNN classifier
$C = \{C_1, C_2, \ldots, C_M\}$	The labeled sets for all classes
$\tilde{y}_i$, $\hat{y}_i$	$\tilde{y}_i$ is pseudo-label obtained from clustering results. $\hat{y}_i$ is the predicted results from the classifier used in poisoned node selection strategy
$\mathbf{X}^L$, $\mathbf{X}^U$	The embedding matrices for labeled ($\mathbf{X}^L$) and unlabeled nodes ($\mathbf{X}^U$)
K	The number of clusters
$V_P^{Primary}, V_P^{Check}, V_{Poison}$	$V_P^{Primary}$ denotes the set of nodes that pass the initial selection. V_P^{Check} denotes the set of nodes that pass dual-ranking reselection. V_{Poison} denotes the final chosen set of poisoned nodes
ψ	Threshold for connecting edge operations based on cosine similarity
φ	Threshold for $D - value$
cfd_{v_i}	The confidence score of node v_i
$f_t(v_i, T)$	Trigger builder, v_i is the poisoned node, and T is the trigger size
H	Number of neighbor hops for node v_i
V_t	The set of nodes in the trigger

previously mentioned symbols, Table 1 lists some additional symbols used in Section 3. The upper part of Table 1 includes symbols that will appear in the 'poisoned node selection strategy', while the lower part contains symbols relevant to the "trigger builder".

Problem Definition. In the backdoor attack on the node classification task, v_i represents the ith node, and $y_{ground-truth}$ is the true label of v_i. $f_t(v_i, T)$ is the trigger builder, where v_i is the poisoned node and T is the size of the trigger. y_{target} is the attacker's predefined target class. The attacker's goal is to force the classifier $f_\theta(\cdot)$ to classify the nodes with the trigger into the attacker's predefined target class while maintaining normal performance on nodes without the trigger. The attack is formulated as follows:

$$\begin{cases} f_\theta(v_i) = y_{ground-truth}, \\ f_\theta(f_t(v_i, T)) = y_{target}. \end{cases} \tag{1}$$

2.2. Threat model

After introducing the notation and problem definition, this subsection presents the threat model of this paper. Following previous work [16], the threat model consists of two main components: the attacker's goals and the attacker's capability.

Attacker's goal: To mislead the classifier into classifying nodes with triggers as the attacker's chosen class, while ensuring the classifier performs normally on nodes without triggers.

Attacker's capability: During the training phase, the attacker can manipulate the training dataset by attaching triggers and modifying the labels of nodes without knowing any information about the downstream classifiers, such as their structure and parameters. Triggers can be inserted into the input graph during the test phase.

2.3. Preliminary analysis

In this subsection, we take the classical node classification dataset Cora as an example to analyze the issue of imbalanced ASR in the backdoor attack method (UGBA [15]), which is specifically designed for node classification tasks. As shown in Table 2, this dataset is characterized by a typical class distribution imbalance.

In the backdoor attack on the node classification task, during the training phase, the attacker attempts to select unlabeled non-target class nodes as poisoned nodes, attaching triggers and target labels to them. We postulate that in order to maintain a balanced distribution of ASRs among various classes, the attacker must endeavor to choose nodes with equal care from each non-target category. Besides, the quantity of nodes selected from each class should be moderate. To validate our hypothesis, we conduct an analysis of the ASR for each class within the Cora dataset. This examination is complemented by an assessment of the class distribution among the selected nodes that are poisoned. Specifically, we fix the poisoning ratio at 0.01, employ the GraphSAGE [8] as the underlying model architecture, and designate the class of interest for the attacker as 0. The comprehensive overview of the parameters employed can be found in Section 4.1.4. We have the following observations:

Firstly, Table 2 displays the ASR for each non-target class, along with the proportion of nodes from each class relative to the total number of nodes in the Cora dataset. It is clear that nodes in Class 6 exhibit the highest ASR at 99.05%, whereas nodes in class 2 have the lowest ASR at 67.32%, resulting in a difference of 31.73%. Despite class 3 and class 4 nodes achieving ASRs of 86.32% and 86.49%, respectively, these values are still significantly lower than that of class 6, indicating a clear imbalance in ASRs.

Secondly, Fig. 2 illustrates our analysis of all poisoned nodes in the training phase of the Cora dataset. According to the reported results, we can make the following in-depth observations and analysis:

- **Inaccuracy in Selection of Poisoned Nodes**: Fig. 2 demonstrates that attacker designates a total of 21 nodes from non-target classes as poisoned nodes, relying on classification results. However, this strategy inadvertently includes approximately 14.29% of nodes from the target class. Furthermore, the absence of any poisoned nodes from class 1 contributes to the notably low ASR for that class, as observed in Table 2.
- **Improper Allocation of Poisoned Nodes**: Fig. 2 illustrates the distribution of poisoned nodes across all classes from 0 to 6, with the number of nodes per class being (3, 0, 1, 4, 5, 6, 2). Despite class 5 nodes comprising only 11% of the total nodes (ranking fourth in prevalence) in the original dataset, they constitute the largest group among the poisoned nodes. Conversely, class 3 nodes, which make up 30.21% of the original dataset (the highest proportion), are represented by the third-largest number of poisoned nodes.

To sum up, these experimental findings strongly support our hypothesis that the selection of poisoned nodes significantly contributes to the observed imbalance in ASRs. Let's delve deeper into the process of selecting poisoned nodes. During the training phase, the attacker faces the challenge of accurately identifying and selecting unlabeled nodes from each non-target class to serve as poisoned nodes. These nodes are then modified with triggers and labeled with the target label. At this stage, the attacker lacks knowledge of the true class of these nodes and thus relies on a classifier to categorize them. Based on the classification results, the attacker proceeds to select poisoned nodes from each class. However, in the presence of data imbalance, the accuracy of the attacker's classifier is compromised. Consequently, the poisoned node selector, which relies on these classification results, fails to accurately choose poisoned nodes

Table 2
ASR for non-targeted class (Cora).

Class	1	2	3	4	5	6
ASR	95%	**67.32%**	86.32%	86.49%	91.11%	**99.05%**
Distribution	8.01%	15.44%	**30.21%**	15.73%	11%	**6.65%**

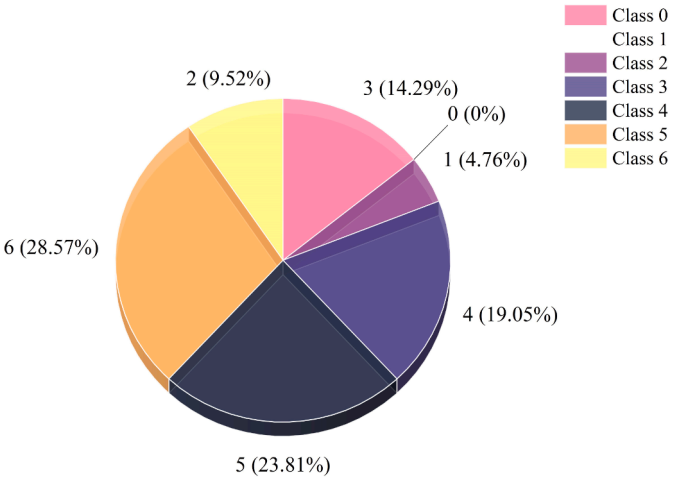


Fig. 2. Class distribution of the poisoned nodes.

from each non-target class. Moreover, the selection of nodes from each class often deviates from what is considered reasonable, as more nodes should ideally be chosen from classes with a larger number of nodes.

In order to ensure a more reasonable process for selecting poisoned nodes, we develop a novel strategy (see Section 3.2) that aims to accurately identify such nodes from each non-target class. This strategy consists of three essential components: initial selection of poisoned nodes, dual-ranking reselection, and imbalanced filtering.

3. Attack framework

3.1. Attack overview

This section delves into the mechanics of the Balanced Poisoning Backdoor Attack (BPBA). As illustrated in Fig. 3, BPBA comprises two key components: a poisoned node selection strategy and an adaptive trigger builder. The strategy for selecting poisoned nodes serves as the initial phase of our attack methodology. During this phase, we process the input graph and leverage a novel metric-based approach to meticulously pinpoint poisoned nodes across all categories except the target class. This ensures a diverse and effective selection of nodes to maximize the impact of the backdoor. Following this, the adaptive trigger builder is designed to craft stealthy triggers specifically for the selected poisoned nodes. To maintain a subtle presence within the dataset, we adhere to the principle of connecting edges between similar nodes, allowing the triggers to blend seamlessly with legitimate data.

In the subsequent subsections, we elaborate on these two integral components of BPBA, highlighting their unique contributions to the attack's efficacy, balance and stealth.

3.2. Poisoned node selection strategy

This section delineates the poisoned node selection strategy, structured into three stages: *initial selection of poisoned nodes*, *dual-ranking reselection*, and *imbalanced filtering*.

Firstly, the initial selection of poisoned nodes phase identifies a pool of potential poisoned nodes based on predefined criteria. Secondly, the dual-ranking reselection phase refines the initial selection using a dual-ranking system. Finally, the imbalanced filtering phase rejects nodes that lie on multiple class boundaries in the embedding space, ensuring a balanced distribution of poisoned nodes in the non-target classes.

Each of these stages is designed to complement the others, resulting in a meticulously curated set of poisoned nodes optimally positioned to execute the backdoor attack without raising suspicion. In the following subsections, we discuss each stage in detail, explaining the rationale and methodology behind each step.

3.2.1. Initial selection of poisoned nodes

As discussed in Section 2.3, we observe that imbalanced data can lead to uneven ASRs across different classes in node classification problems targeted by backdoor attacks. This discrepancy may arise because the selection of poisoned nodes within each non-target class is often based on the classifier's prediction results. However, in the context of imbalanced data, these predictions can be unreliable. Kang et al. Kang et al. [17] have demonstrated that a classifier trained on an imbalanced dataset is particularly vulnerable to performance degradation. This inherent limitation of supervised methods under data imbalance motivates our integration of unsupervised techniques, forming a hybrid approach to overcome the one-sidedness of relying solely on a single paradigm.

To address this issue, we propose a hybrid approach that leverages both supervised and unsupervised algorithms for the initial selection of poisoned nodes, aiming to reduce the prediction bias inherent in imbalanced datasets. Unsupervised learning algorithms are particularly useful in this context, as they can uncover hidden patterns and relationships within node embeddings without relying on labels, thus mitigating the bias that supervised methods may introduce.

In our method, we first derive the classification labels $\hat{y}_i$ using the GNN encoder. Next, we cluster the high-quality embeddings produced by the GNN encoder to generate pseudo-labels, denoted as $\tilde{y}_i$. Finally, we compare pseudo-labels $\tilde{y}_i$ with the predictions $\hat{y}_i$. Nodes whose pseudo-labels align with the supervised predictions are selected as part of the initial poisoned node set $V_P^{Primary}$. The following part provides a detailed explanation of this process.

To represent the embeddings of labeled and unlabeled nodes, we use $\mathbf{X}^L \in \mathbb{R}^{|V^L| \times d}$ and $\mathbf{X}^U \in \mathbb{R}^{|V^U| \times d}$, respectively. Here, d denotes the embedding dimension, $|V^L|$ / $|V^U|$ represents the number of labeled / unlabeled nodes. Once the embeddings are obtained, we apply the K-Means clustering algorithm to the embeddings of the unlabeled nodes to compute the cluster centers, as shown in Eq. (2).

$$\{B_1, B_2, \ldots, B_K\}, \{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K\} = K - Means(\mathbf{X}^U), \tag{2}$$

where B_i represents the ith cluster, $\mathbf{c}_i$ is the cluster center of the ith cluster, and K is the hyperparameter.

Once the clustering results for the unlabeled nodes are determined, the next step is to assign a pseudo-label corresponding to the true class to each cluster. To accomplish this task, we begin by computing the mean embeddings for the labeled nodes within the training set, yielding the embedding centers for each class: $\{\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2, \ldots, \hat{\mathbf{c}}_K\}$. We then iterate over each class center $\hat{\mathbf{c}}_i$ in $\{\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2, \ldots, \hat{\mathbf{c}}_K\}$ and compare it with each cluster center $\mathbf{c}_j$ in $\{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K\}$. The nearest cluster center $\hat{\mathbf{c}}_r$ (where $r \in \{1, 2, \ldots, K\}$) is identified using the Euclidean distance function, and the corresponding pseudo-label is assigned to $\mathbf{c}_j$. All nodes within $\mathbf{c}_j$ are then labeled accordingly. This process yields pseudo-labels for each non-target node. The nearest cluster center $\hat{\mathbf{c}}_r$ of $\mathbf{c}_j$ is determined as follows:

$$\hat{\mathbf{c}}_r = \underset{i}{\operatorname{argmin}}\left(dist\left(\hat{\mathbf{c}}_i, \mathbf{c}_j\right)\right), i, j, r \in \{1, 2, \ldots, K\}, \tag{3}$$

$$\tilde{y}_{\mathbf{c}_j} = \hat{y}_{\hat{\mathbf{c}}_r}, \tag{4}$$

where $dist(\cdot)$ represents the Euclidean distance function, $\mathbf{c}_r$ is the cluster center in $\{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K\}$ that is closest to $\hat{\mathbf{c}}_i$. Subsequently, we use the classifier $f_\theta(\cdot)$ to obtain the predicted class $\hat{y}_i$ for each unlabeled node v_i.

For each unlabeled node v_i, we now have two labels: $\hat{y}_i$ from the GNNs classifier and $\tilde{y}_i$ from the pseudo-labeling process. We retain only

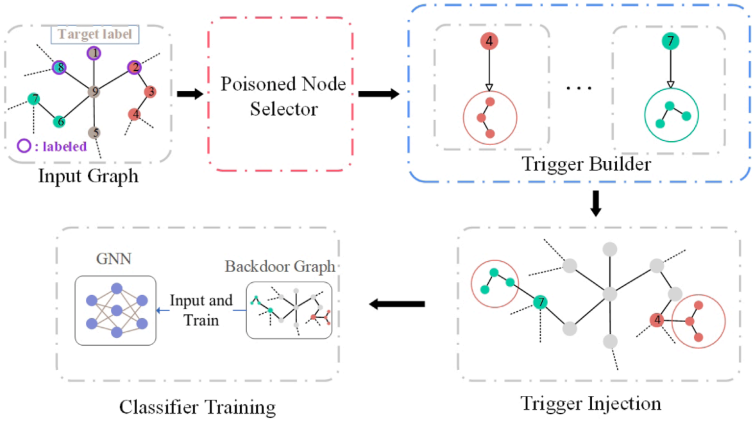


Fig. 3. The overall process of BPBA.

those nodes where the labels match and add them to the preliminary selection set $V_P^{Primary}$. This set forms the basis for the initial selection of poisoned nodes, ensuring a balanced representation across classes and reducing the bias inherent in imbalanced datasets.

3.2.2. Dual-ranking reselection

Once the preliminary selection of poisoned nodes is complete, we proceed with the dual-ranking reselection process, utilizing both spatial and confidence rankings to refine the selection. This approach leverages the strengths of unsupervised spatial ranking and supervised confidence ranking to enhance the quality of the poisoned node set.

Spatial Ranking. For each unlabeled node v_i belonging to class r with embedding $\mathbf{x}_i^U$, we calculate the distance between v_i and its corresponding class center $\hat{\mathbf{c}}_r$. We then sort the nodes in $V_P^{Primary}$ that belong to class r based on this distance to prioritize those that are closer to the class center.

$$d_i = dis\left(\mathbf{x}_i^U,\ \hat{\mathbf{c}}_r\right), \tag{5}$$

$$(v_1, v_2, \dots v_M) = Fun\left(sort\left(\{d_1, d_2, \dots, d_M\}\right)\right), \tag{6}$$

where d_i represents the distance of v_i from the center $\hat{\mathbf{c}}_\mathbf{r}$ of class r, $sort(\cdot)$ is the sorting function with ascending order, and $Fun(\cdot)$ retrieves the corresponding node sequence based on the sorted distance sequence. The sorting resulting $(v_1, v_2, \dots v_M)$ contains class r nodes from $V_P^{Primary}$ ranked by spatial proximity.

Confidence Ranking. For every node v_i in $V_P^{Primary}$, we convert the GNNs' output into a confidence score, indicating the classifier's certainty in its prediction.

$$Cfd_{v_i} = \max\left(softmax(\mathbf{output})\right), \tag{7}$$

where $\mathbf{output}^{1\times K}$ is the output vector of GNNs. The output vector is transformed into a confidence score Cfd_{v_i}, which reflects the classifier's confidence in the pseudo-label for v_i. We sort the nodes in $V_P^{Primary}$ that belong to class r in non-increasing order of confidence to obtain $\left(v'_1, v'_2, \dots v'_M\right)$.

$$(v'_1, v'_2, \dots v'_M) = Fun\left(sort\left(\mathbf{output}_1, \mathbf{output}_2, \dots, \mathbf{output}_m\right)\right). \tag{8}$$

Ranking Combining. By combining spatial and confidence rankings, we generate two sequences for each class in $V_P^{Primary}$: $(v_1, v_2, \dots v_M)$ and $(v'_1, v'_2, \dots v'_M)$. To effectively synthesize these rankings, we employ the Rank-Biased Overlap (RBO) [18] measure, which quantifies the similarity between two rankings and offers a systematic method for their integration. The RBO score, denoted as sim_A, is calculated by comparing the rankings of nodes in the sets $(v_1, v_2, \dots v_M)$ and $(v'_1, v'_2, \dots v'_M)$. Subsequently, we determine the score for each node, DSC_A, which is the maximum of sim_A and $1 - sim_A$, weighted by the corresponding set of nodes, and the minimum of sim_A and $1 - sim_A$, weighted by the other set. Finally, we sort the nodes with discending order within each class in $V_P^{Primary}$ based on their DSC values and reselect nodes from each class to form a new set V_P^{Check}, representing an improved selection of poisoned nodes.

3.2.3. Imbalanced filtering

Song et al. [19] have identified that in imbalanced datasets, a few nodes frequently exhibit topological imbalance, meaning they are positioned near the boundaries between different classes. For instance, consider nodes v_i and v_j, belonging to classes A and B, respectively. If the embedding of v_j, lies close to the boundary between A and B, it may be incorrectly pseudo-labeled as class A, even if it is geometrically farther from the center of class A than v_i. This can lead to misclassification, as nodes that are proximate to multiple class centers simultaneously may have less reliable pseudo-labels. Consequently, it is imperative to filter out such nodes from V_P^{Check}.

To address this, we compute the distance $sp_{v_i}^{\mathbf{c}_n}/sp_{v_i}^{\mathbf{c}_n^s}$ between the embedding of each node v_i in V_P^{Check} and its nearest/second nearest class center $\mathbf{c}_n/\mathbf{c}_n^s$, respectively. We then define $D-value$ as the absolute difference between these two distances, normalized by the distance to the nearest class center: $D-value = \frac{\left|sp_{v_i}^{\mathbf{c}_n^s} - sp_{v_i}^{\mathbf{c}_n}\right|}{sp_{v_i}^{\mathbf{c}_n}}$. If the $D-value$ is below a threshold φ, the node is considered to have an unreliable pseudo-label and is removed from V_P^{Check}. Nodes with $D-value$ above the threshold comprise our final set of poisoned nodes, denoted as V_{poison}.

Following the generation of V_{poison}, we adjust the number of poisoned nodes per class to reflect the original class distribution. Classes with a higher percentage of nodes in the original data will have a proportionally larger representation in V_{poison}. This step ensures that the poisoned node set maintains a balance that mirrors the original dataset.

The pseudocode for the algorithm is presented in Algorithm 1. The inputs include the dataset G, node attribute matrix $\mathbf{X}$, adjacency matrix $\mathbf{A}$, and the poisoning rate ρ. The output is the poisoned node set V_{poison}. The algorithm is structured into three main sections: lines 3 to 8 cover the 'Initial Selection of Poisoned Nodes', lines 10 to 15 focus on 'Dual-Ranking Reselection', and lines 17 to 21 handle 'Imbalanced Filtering'.

3.3. Trigger builder

Once we have poisoned nodes, we need to build stealthy triggers for them. In this section, we introduce the trigger builder. After obtaining the poisoned nodes V_{poison}, based on the trigger size T, we adaptively design stealth triggers for each poisoned node. Designing triggers involves two main steps: determining the internal structure of the trigger and connecting the trigger to the poisoned node. Therefore, we divide the design of the trigger into two parts: **the internal structure of the trigger** and **the connection of the trigger to the poisoned node**. As

Algorithm 1: Poisoned node selection strategy.

Input: Graph $G = (V, E, \mathbf{X}, \mathbf{A})$, V^L represents labeled nodes, V^U represents unlabeled nodes, and $V^L \cup V^U = V$, poisoning rate ρ, GNN model f;

Output: $V_{poision}$;

$V_{poision} = \{\ \}$;
// **1.Initial Selection of Poisoned Nodes**
Train f based on V^U;
Obtain embedding matrices for labeled and unlabeled nodes $\mathbf{X}^L$, $\mathbf{X}^U$;
Obtain clusters $\{B_1, B_2, \ldots, B_K\}$ and cluster centers $\{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K\}$ via Eq. (2);
Obtain the embedding centers for each class $\{\hat{\mathbf{c}}_1, \hat{\mathbf{c}}_2, \ldots, \hat{\mathbf{c}}_K\}$;
Get pseudo-label $\tilde{y}_{\mathbf{c}_i}$ for each cluster, which in turn gives us the pseudo-label $\tilde{y}_i$ for each node via Eqs. (3) and (4);
Get predicted label $\hat{y}_i$ for each unlabeled node by f to form $V_P^{Primary}$;
// **2.Dual-ranking Reselection**
for each class in $V_P^{Primary}$ do:
    Get nodes of class r in $V_P^{Primary}$;
    Obtain $(v_1, v_2, \ldots, v_M)$ via Eqs. (5) and (6);
    Obtain $(v'_1, v'_2, \ldots, v'_M)$ via Eqs. (7) and (8);
    Calculate the *sim* value for two sort results $(v_1, v_2, \ldots, v_M)$ and $(v'_1, v'_2, \ldots, v'_M)$;
    Calculate the DSC values for nodes in class r to form V_P^{Check}.
// **3.Imbalanced Filtering**
for v_i in V_P^{Check} do:
    Get the distance $\mathrm{sp}_{v_i}^{\mathbf{c}_n}$ / $\mathrm{sp}_{v_i}^{\mathbf{c}_n^s}$ of node v_i from the nearest / second nearest class center $\mathbf{c}_n$/$\mathbf{c}_n^s$;
    Calculate $D-value$ using $\mathrm{sp}_{v_i}^{\mathbf{c}_n}$ and $\mathrm{sp}_{v_i}^{\mathbf{c}_n^s}$ based on $D-value = \frac{\left|\mathrm{sp}_{v_i}^{\mathbf{c}_n^s} - \mathrm{sp}_{v_i}^{\mathbf{c}_n}\right|}{\mathrm{sp}_{v_i}^{\mathbf{c}_n}}$;
    If $D-value > \varphi$:
        Add v_i to $V_{poision}$;
return $V_{poision}$;

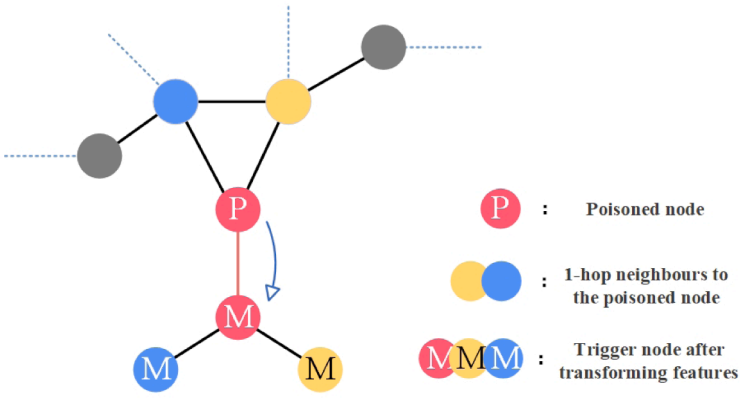


Fig. 4. The overall process of trigger builder.

shown in Fig. 4, the nodes labeled with the letter 'M' are the internal nodes of the trigger, and the red connecting lines represent the connections between the trigger and the poisoned node (Algorithm 1).

Internal Structure of the Trigger. Designing the internal structure of the trigger involves three key considerations: nodes, features, and structure. We will discuss these three aspects in detail.

- **Nodes**: We choose the poisoned node itself and its $T-1$ H-hop neighbors as trigger nodes. (Both T and H are controlled by hyper-parameters.)
- **Features**: After selecting the trigger nodes, we perform a feature transformation on the features $\mathbf{x}_i$ of each node, as shown in Formula (9).

$$\tilde{\mathbf{x}}_i = \sigma\left(\mathbf{W}_f \mathbf{x}_i + \mathbf{b}_f\right), \tag{9}$$

  where $\mathbf{x}_i$ is the original feature, $\mathbf{W}_f$ and $\mathbf{b}_f$ are learnable parameters, $\sigma(\cdot)$ is the activation function, and $\tilde{\mathbf{x}}_i$ is the transformed node feature.
- **Structure**: To ensure the stealthiness of the trigger, we calculate the cosine similarity between the nodes inside the trigger, as shown in Formula (10). If the similarity exceeds the threshold ψ, an edge is added. Let V_i be the set of nodes in the trigger, including the poisoned node v_p and its $T-1$ H-hop neighbors. The calculation of the cosine similarity between the nodes in the trigger is as follows:

$$\mathrm{sim}(v_i, v_j) = \frac{\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{x}}_j}{\|\tilde{\mathbf{x}}_i\| \|\tilde{\mathbf{x}}_j\|}. \tag{10}$$

Connection of the Trigger to Poisoned Node. After completing the construction of the trigger's internal structure, we also need to connect the trigger to the poisoned node to complete the implantation process of the trigger. Existing research [11] shows that using simple defense strategy, for example, pruning edges linking nodes with low cosine similarity, has a strong defensive effect. Similar to the 'structure' described above, we calculate the similarity between the nodes inside the trigger and the poisoned nodes and connect the node with the highest similarity to the poisoned node.

By following these steps, we can create stealthy and effective triggers for the poisoned nodes, ensuring that the backdoor attack remains hidden while maintaining the functionality of the target model.

3.4. Model optimization

Supervised Loss. Following the implementation of the 'poisoned node selection strategy' and 'trigger builder', we have identified the poisoned nodes and crafted triggers for each of them. The supervised loss function (L_{sup}) is then defined to optimize the classifier's performance. L_{sup} is based on the cross-entropy loss, which optimizes the performance of classification model.

Stealth Loss. To ensure that the triggers are both stealthy and robust, the trigger builder focuses on connecting poisoned nodes within the trigger with high cosine similarity. Additionally, it ensures that the internally connected nodes within the trigger also exhibit high similarity. A stealth loss is introduced to penalize the trigger builder for connections that do not meet the specified similarity threshold as follows:

$$\min_{\theta_{f_t}} L_{\text{stealth}} = \sum_{v_i \in V_{\text{poison}}} \sum_{(v_j, v_l) \in \beta_P^i} \max(0, \psi - \mathrm{sim}(v_j, v_l)), \tag{11}$$

where β_P^i represents all edges connecting poisoned nodes with the trigger and inside the trigger, ψ is the similarity threshold, and θ_{f_t} represents the parameters of the trigger builder $f_t(\cdot)$. The stealth loss function encourages the trigger builder to maintain high similarity both between the poisoned nodes and the trigger nodes, and among the trigger nodes themselves, thereby enhancing the stealthiness of the attack.

By optimizing both the supervised and stealth losses, we aim to achieve a balance between classification accuracy and the stealthiness of the backdoor attack, ensuring the triggers remain inconspicuous while still being effective in compromising the model.

4. Experiments

In this section, we present the experimental setup and results for BPBA on four multi-scale and imbalanced datasets. Our experiments aim to address the following research questions:

Table 3
Statistics of the datasets.

Datasets	#Nodes	#Edges	#Feature	#Class
Cora	2708	5429	1443	7
Pubmed	19,717	44,338	500	3
Flickr	89,250	899,756	500	7
OGB-arxiv	169,343	1,166,243	128	40

Table 4
Proportion of nodes of each class to the total number of nodes.

	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6
Cora	12.96%	8.01%	15.44%	**30.21%**	15.73%	11%	**6.65%**
Pubmed	**20.81%**	39.25%	**39.94%**	–	–	–	–
Flickr	5.90%	9.53%	7.19%	5.49%	25.73%	**3.90%**	**42.26%**

- RQ1: Does BPBA achieve effective, balanced, and stealthy attack effects compared to other baseline methods?
- RQ2: How do poisoning rate, trigger size and parameter H affect the efficacy of backdoor attacks?
- RQ3: How do poison node selection strategy and trigger builder affect attack performance?

The remainder of this section will delve into the experimental methodology, including the datasets used, the evaluation metrics employed, and the comparison with baseline methods. We will then present the results for each research question, discussing the implications of our findings.

4.1. Experimental settings

4.1.1. Data descriptions

To evaluate the efficacy of BPBA, we conduct experimental evaluations on four benchmark datasets: Cora [20], Pubmed [20], Flickr [21], and OGB-arxiv [22]. Table 3 summarizes the key characteristics of these datasets, while Table 4 details the distribution of nodes across different classes in each dataset, expressed as a proportion of the total number of nodes. For the OGB-arxiv dataset, which encompasses a vast array of classes, we report the proportions of nodes from the most and least common classes to maintain brevity. Notably, Class 16 in OGB-arxiv constitutes the largest proportion of nodes at 16.13%, in stark contrast to Class 12, which is the least represented with a mere 0.01% of the dataset.

- **Cora**: This dataset contains information about scientific papers and is commonly used for text classification tasks. It consists of 2708 nodes, with each node representing a scientific paper. These nodes are categorized into 7 classes.
- **Pubmed**: The Pubmed dataset comprises 19,717 papers extracted from the Pubmed database, focusing on diabetes-related research. These papers are divided into 3 classes. Each paper is represented as a node in the dataset, and links between nodes indicate citation relationships between papers.
- **Flickr**: The Flickr dataset originates from the NUS-WIDE project. The SNAP website gathers Flickr data from various sources, including NUS-WIDE, and constructs an undirected graph. This graph consists of 89,250 nodes categorized into 7 classes. Each node represents an image uploaded to Flickr. If two images share common properties, such as the same geographical location or comments from the same user, an edge is created between their corresponding nodes.
- **OGB-arxiv**: OGB-arxiv is an academic citation network where each node represents a research paper, and links between nodes indicate citation relationships. Each paper is associated with a 128-dimensional feature vector, obtained by averaging word embeddings from its title and abstract.

These diverse datasets allow us to evaluate BPBA across different graph structures and application domains, providing a comprehensive understanding of its performance in various scenarios.

4.1.2. Baselines

To demonstrate the effectiveness of BPBA, we compare it with a variety of backdoor attack methods and GIA methods. The selected backdoor attack baselines are BATGNN-Gen [16], GTA [23], and UGBA [15]. The selected GIA baselines are TDGIA [24] and AGIA [11]. These methods are selected because they represent different approaches to attack methods, some specifically tailored for node-level backdoors and others for large-scale datasets.

- **BATGNN-Gen** [16]: A variant of the classic BATGNN method, which uses the ER model to generate triggers. The key modification is that the features of the initial trigger nodes are sampled from a Gaussian distribution. Triggers are then injected into randomly selected poisoned nodes.
- **GTA** [23]: This method defines triggers as specific subgraphs that include both topological structures and descriptive features, offering a wide range of design options. GTA dynamically adjusts triggers to fit different graphs, enhancing attack effectiveness and stealth. It does not require knowledge of downstream models or fine-tuning strategies.
- **UGBA** [15]: As the SOTA backdoor attack method for GNNs, UGBA introduces a poisoning node selection method that selects representative nodes from each non-target class as poisoned nodes. It then adaptively generates the corresponding trigger and its features for each poisoned node.
- **DPGBA** [25]: By integrating distribution detection and adversarial learning, it ensures the triggers are difficult to detect, while employing specialized modules to enhance the model's memorization of triggers, achieving a high attack success rate even under various defense mechanisms.
- **SPERA** [26]: It is a highly stealthy graph backdoor attack method that operates without modifying the graph structure, relying solely on perturbing node attributes. By leveraging global importance-driven feature selection and a neighborhood-aware semantic trigger generation mechanism, it maintains high attack effectiveness while effectively evading existing defense measures.
- **TDGIA** [24]: This method introduces the GIA scenario, where attackers cannot modify the existing input graph structure or node features. Instead, they inject adversarial nodes to launch attacks. TDGIA assesses the topological vulnerability of the input graph to select poisoned nodes and uses a smooth feature optimization objective to generate features for the injected nodes.
- **AGIA** [11]: Similar to TDGIA, AGIA follows the GIA attack scenario. It introduces the Harmonious Adversarial Objective (HAO) to ensure the homogeneity distribution of the original graph and uses gradient information to optimize the features of the injected nodes.

To ensure the stealthiness and robustness of the attack, we conduct experiments in a prune defense setting. Many existing backdoor attack methods are not stealthy because they create edges between nodes with low cosine similarity during trigger injection. Prune defense aims to defend against such attacks by removing edges between nodes with low cosine similarity. This setting allows us to evaluate the robustness of the attack methods against common defense mechanisms.

4.1.3. Evaluation metrics

In this article, we employ two key evaluation metrics: **Attack Success Rate (ASR)** and **Clean Accuracy (CA)**. These metrics provide insights into the effectiveness and stealthiness of the backdoor attack methods.

- ASR measures the proportion of poisoned nodes that are successfully classified into the target class by the model. It is calculated by dividing the number of successfully classified poisoned nodes $N_{success}$ by

the total number of poisoned nodes N_P. A higher ASR indicates a more effective attack.

$$\text{ASR} = \frac{N_{success}}{N_P}. \tag{12}$$

- CA evaluates the classification accuracy of the classifier on clean nodes (nodes that have not been attacked). CA serves as a direct metric for evaluating model performance on non-target samples. A high CA value signifies that the model's core functional parameters remain intact and that its decision boundaries are preserved within the clean input domain. This confirms that the trigger's presence and activation are localized and well-regulated-they only influence model behavior under specific trigger-embedded conditions without perturbing its reasoning process for standard inputs.

$$\text{CA} = \frac{\sum_{i=1}^{m} \mathbb{I}(f_\theta(v_i) = y_{\text{ground-truth}})}{m}, \tag{13}$$

where $\mathbb{I}(\cdot)$ is an indicator function that returns 1 if the condition inside is true and 0 otherwise, $f_\theta(v_i)$ is the classifier's prediction for node v_i, and $y_{\text{ground-truth}}$ is the true class label of the node.

By analyzing these metrics, we can assess the trade-off between the attack's effectiveness and stealthiness, which is crucial for backdoor attack.

4.1.4. Parameter settings

Our attack strategy consists of two primary components: the poisoned node selection strategy and the trigger builder. For the poisoned node selection strategy, we employ the following parameters for each dataset: The encoder used is a Graph Convolutional Network (GCN) with a hidden layer size of 32, and we set the Rank-Biased Overlap (RBO) parameter to $p = .9$. In the trigger builder phase, we set the neighbor distance (T) to 1 and the trigger size to 3. The attacker selects the target class $l_{\text{target}} = 0$, and the poisoning rate is set at 0.01. For testing, we use the GraphSAGE [8] model with a learning rate of 0.01.

We randomly mask 20% of the nodes from the original dataset. Of these masked nodes, half are designated as target nodes for assessing the attack performance, while the remaining half serve as clean test nodes to gauge the prediction accuracy of backdoored models on unaltered samples. The graph consisting of the remaining 80% of nodes is utilized as the training graph. The final outcome is derived from the average results of three independent runs of the BPBA approach. All experiments are conducted on a system equipped with an NVIDIA RTX 3090 GPU, an Intel Xeon E5-2640 CPU, and running Linux CentOS 7.1.

4.2. Experimental results (RQ1)

To address Research Question 1, we conduct a comparative analysis of BPBA against backdoor attack and GIA baseline methods on four multi-scale imbalanced datasets under two settings (defended and undefended). Specifically, we employ the pruning defense setting because many existing backdoor attack methods lack stealthiness when injecting triggers, as they create edges with low cosine similarity between nodes. Pruning defense aims to defend against such attacks by removing edges with low cosine similarity between nodes. This setting allow us to assess the robustness of the attack methods against common defense mechanisms. Additionally, to demonstrate the balanced ASR of our method, we compare the ASR of each class for BPBA with the SOTA method for node classification tasks, UGBA.

First, we summary the average results of backdoor attacks in Tables 5 and 6. From these results, we can draw the following conclusions:

- **BATGNN-Gen**: The method demonstrates poor performance across all four datasets under both GCN and GraphSAGE architectures. For instance, on the GraphSAGE architecture and the no-defense setting, the ASR for Flickr and OGB-arxiv datasets are 0 and 51.27%, respectively. This indicates that the random selection of poisoned nodes and trigger generation does not lead to effective attack outcomes, especially on large-scale datasets. Meanwhile, on the GraphSAGE architecture and the prune defense setting, BATGNN-Gen's performance decreases by 38.01% on Cora and 48.74% on Pubmed, further underscoring the lack of stealth in its triggers.
- **GTA**: GTA outperforms BATGNN-Gen on most datasets under both GCN and GraphSAGE architectures, showcasing its effectiveness in selecting poisoned nodes and dynamically adapting triggers to different poisoned nodes. However, neither GTA nor BATGNN-Gen perform well under the prune defense setting, emphasizing the importance of designing stealthy triggers.
- **UGBA**: Under both GCN and GraphSAGE architectures, UGBA demonstrates better overall performance compared to BATGNN-Gen and GTA, showcasing the benefits of selecting representative nodes from each non-target class to increase ASR. For example, under the GraphSAGE, UGBA achieves excellent results on both the Flickr and OGB-arxiv datasets. Its excellent attack performance on large-scale datasets further suggests its suitability for such datasets compared to BATGNN-Gen and GTA. Under the prune defense setting, UGBA maintains strong and stable attack performance, again highlighting the significance of stealthy trigger design.
- **DPGBA**: Under both GCN and GraphSAGE frameworks and no-defense settings, DPGBA achieves outstanding attack performance, attaining average ASR of 96.1% and 94.03% across the four datasets, respectively. Meanwhile, its CA remains comparable to that on clean graphs, further demonstrating its exceptional attack stealth. However, under the prune defense setting, DPGBA exhibits significant ASR degradation on both architectures, indicating its inadequate adaptability to this specific defense strategy.
- **SPEAR**: This method achieves remarkably effective attack performance across multiple datasets, under both GCN and GraphSAGE architectures. Further observation reveals that the SPEAR method maintains competent attack effectiveness under both prune defense and no-defense settings, demonstrating its strong attack robustness. Moreover, its classification accuracy (CA) remains close to that attained on clean graphs, reflecting commendable stealth capabilities. However, on large-scale datasets such as OGB-arxiv, its performance proves slightly less competitive.
- **BPBA**: Under the GCN architecture, BPBA achieves either the best or second-best performance on most datasets. Notably, it delivers optimal Attack Success Rate (ASR) under both defense strategies on the Pubmed dataset, and also delivers commendable results on larger-scale datasets. Under the GraphSAGE architecture, BPBA outperforms baseline models on the large-scale OGB-arxiv dataset and achieves the highest ASR under both defense settings. BPBA's capability to select poisoned nodes from each non-target class and design stealthy triggers is clearly reflected in these outcomes. However, on the Cora dataset, BPBA exhibits slightly inferior attack performance compared to the strongest baseline across both defense settings and both architectures. We hypothesize that this phenomenon arises from a fundamental trade-off between attack efficacy and stealth. In densely structured graphs such as Cora, artificially inserted triggers can become particularly noticeable. Consequently, our method prioritizes stealth to enhance robustness against defense mechanisms, a design choice that may marginally reduce attack performance in undefended environments.

These results underscore the effectiveness of BPBA in achieving a targeted backdoor attack while maintaining balance and stealth, especially in the context of large-scale and imbalanced datasets.

Second, we compare BPBA with two SOTA node injection attack methods–TDGIA and AGIA. Since TDGIA and AGIA are designed for large-scale datasets, we perform experiments on the OGB-arxiv dataset using RobustGCN [27] as a test model. From Table 6, we can draw the following conclusions:

Table 5
Overall ASR against baseline on GraphSAGE (↑ ASR|CA, '↑' indicates that a larger ASR value signifies a better attack effect.).

Datasets	Defence	Clean Graph	BAT-GN-Gen	GTA	UGBA	DPGBA	SPEAR	BABP (ours)
Cora	None	0.8202	0.6701| 0.8341	0.9432| 0.8259	0.9306| 0.8400	0.9770| 0.8333	**0.9885**| 0.8396	0.9515| 0.8378
	Prune	0.7876	0.2362| 0.8356	0.1911| 0.8333	0.9712| 0.8296	0.7320| 0.8063	**0.9778**| 0.8266	0.9172| 0.8111
Pubmed	None	0.8496	0.5946| 0.8483	0.9233| 0.8476	0.8970| 0.8521	0.9230| 0.8500	0.9503| 0.8508	**0.9787**| 0.8521
	Prune	0.8513	0.2170| 0.8490	0.2163| 0.8495	0.9142| 0.8528	0.3911| 0.8296	0.9670| 0.8524	**0.9685**| 0.8533
Flickr	None	0.4315	0.0000| 0.4117	**0.9998**| 0.4086	0.9988| 0.4114	0.9880| 0.4640	**0.9998**| 0.4062	0.9633| 0.4108
	Prune	0.4204	0.0000| 0.4115	0.0011| 0.4042	**0.9979**| 0.4040	0.4877| 0.4361	0.9813| 0.4030	0.9889| 0.4041
OGB-arxiv	None	0.6317	0.5427| 0.6362	0.7027| 0.6083	0.9578| 0.6251	0.9560| 0.6580	0.9695| 0.6691	**0.9784**| 0.6319
	Prune	0.6278	0.0651| 0.6217	0.0043| 0.6229	0.9305| 0.6102	0.3077| 0.6726	**0.9866**| 0.6483	0.9492| 0.6156

Table 6
Overall ASR against baseline on GraphSAGE (↑ ASR|CA, '↑' indicates that a larger ASR value signifies a better attack effect.).

Datasets	Defence	Clean Graph	BAT-GN-Gen	GTA	UGBA	DPGBA	SPEAR	BABP (ours)
Cora	None	0.8178	0.6052| 0.8267	**0.9911**| 0.8178	0.8627| 0.8252	0.9530| 0.8330	0.9826| 0.8407	0.9077| 0.8274
	Prune	0.7785	0.2251| 0.7902	0.1971| 0.8104	0.8686| 0.8356	0.6273| 0.7963	**0.9557**| 0.7815	0.8753| 0.8200
Pubmed	None	0.8577	0.7052| 0.8584	**0.9976**| 0.8605	0.8573| 0.8581	0.9180| 0.8510	0.9795| 0.8493	0.7846| 0.8438
	Prune	0.8623	0.2178| 0.8594	0.2184| 0.8629	0.8834| 0.8654	0.3027| 0.8529	**0.9880**| 0.8487	0.7902| 0.8634
Flickr	None	0.4393	0.0000| 0.4513	0.9832| 0.4382	0.9902| 0.4636	0.9480| 0.4560	**0.9999**| 0.4285	0.9856| 0.4623
	Prune	0.4358	0.0000| 0.4623	0.0000| 0.4321	0.9914| 0.4475	0.0028| 0.4408	**1.0000**| 0.4410	0.9795| 0.4387
OGB-arxiv	None	0.6354	0.5127| 0.6363	0.6689| 0.6373	0.9021| 0.6361	0.9420| 0.6580	0.9299| 0.6681	**0.9641**| 0.6385
	Prune	0.6093	0.0051| 0.6245	0.0026| 0.6288	0.8950| 0.6176	0.3062| 0.6736	0.8961| 0.6692	**0.9262**| 0.6290

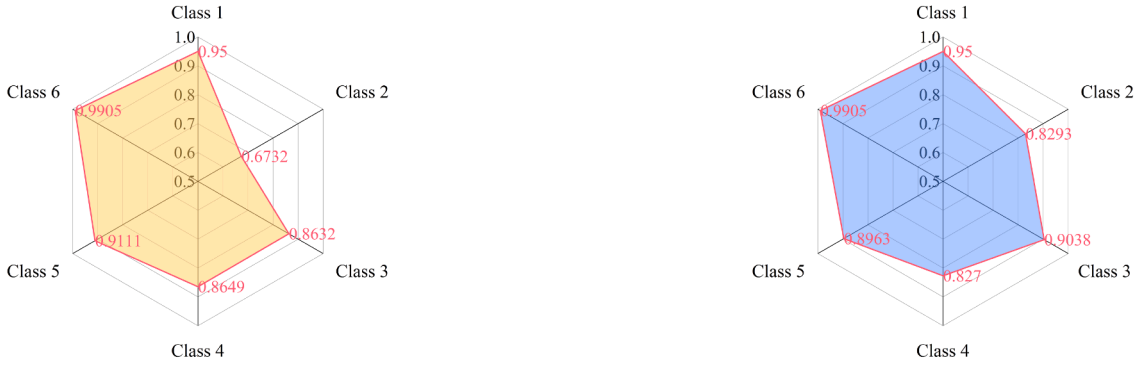
Table 7
Compare BPBA with the node injection attack baseline.

Dataset	TDGIA	AGIA	BABP
OGB-arxiv	74.35%	74.64%	93.25%

- BPBA achieves the best results compared to TDGIA and AGIA. This indicates that BPBA can not only attack common GNN models but also general defense models, achieving good attack effectiveness.
- BPBA achieves a commendable ASR even on large-scale datasets with numerous classes, validating the effectiveness of the poisoned node selection strategy. By strategically selecting unlabeled nodes from each non-target class, BPBA ensures that the attack effect can propagate to every class, thereby enhancing the overall ASR (Table 7).

Third, to evaluate the attack balance of BPBA, we compare its ASR for each class with UGBA, which is specifically designed for node classification tasks. Our experiments are conducted on the Cora dataset under undefended settings. The results, as depicted in Fig. 5, reveal the following insights:

- **Balance of Attack Effectiveness**: BPBA demonstrates a more balanced ASR compared to UGBA. In Class 2, UGBA's ASR is 67.32%, while in Class 1, it reaches 95%, resulting in a 27.68% difference. BPBA, on the other hand, achieves an ASR of 82.93% and 95% for Classes 1 and 2, respectively, reducing the gap to 12.07%. Moreover, the overall variance of ASR for non-target classes is 0.0102 for UGBA and 0.0035 for BPBA, further confirming BPBA's relative balance in attack effectiveness.
- **Accuracy of Poisoned Node Selection**: Jointly analyzing Figs. 2 and 5 together, we observe that both UGBA and BPBA select a total of 21 poisoned nodes based on the classification results. As shown in Fig. 2, out of all the nodes selected by UGBA, 3 are from the target class (Class 0), and none are from Class 1. However, as illustrated in Fig. 5, the poisoned nodes selected by BPBA are from all non-target classes, with only 2 nodes coming from the target class.
- **Reasonable Number of Poisoned Nodes**: From the analysis of Figs. 1 and 5, it is evident that both UGBA and BPBA select poisoned nodes across Classes 0 to 6, with UGBA's selection pattern being (3, 0, 1, 4, 5, 6, 2) and BPBA's being (2, 1, 6, 7, 2, 2, 1). Fig. 2 reveals a discrepancy in UGBA's approach: Class 5 nodes, comprising just 11% of the original dataset, form the largest group among the poisoned nodes, while Class 3 nodes, which represent 30.21% of the dataset, are only the third most numerous. This suggests an unbalanced



(a) UGBA (b) BPBA

Fig. 5. ASR for non-targeted classes by UGBA and BPBA.

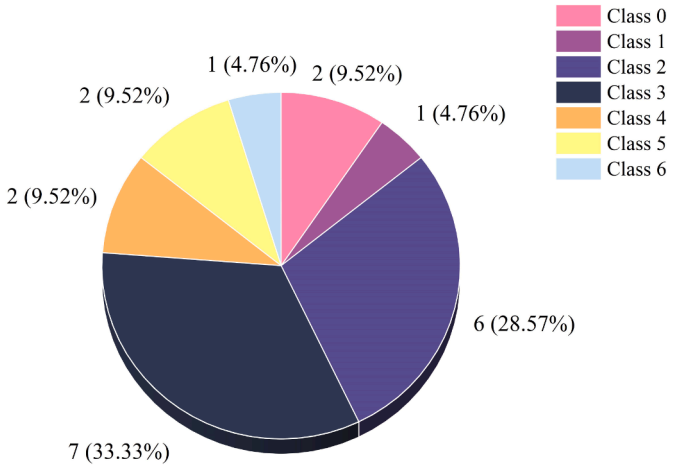


Fig. 6. Class distribution of the poisoned nodes.

distribution by UGBA. In stark contrast, BPBA's selection mirrors the original data's distribution. Class 3 nodes, the most prevalent at 30.21%, are also the most numerous among the poisoned nodes, while Class 6, the least prevalent at 6.65%, corresponds to the smallest number of poisoned nodes. This alignment underscores the rationality of BPBA's node selection strategy (Fig. 6).

These results indicate that BPBA not only selects poisoned nodes accurately but also achieves a balanced ASR across classes, ensuring that the attack's impact is distributed evenly across the dataset. This balanced approach contributes to the stealthiness of the attack, as it reduces the likelihood of detection by defense methods.

Finally, to further verify the stealthiness of BPBA, we conduct an additional evaluation of its impact on clean nodes. Following the data partition scheme in Section 4.1.4, we use the held-out clean test set (10% of each dataset) for this evaluation. The experimental setup focuses on the non-target class nodes within this clean test set. Two models are trained for comparison: clean model (Φ_{Clean}) trained exclusively on the clean training graph, and a backdoored model ($\Phi_{Backdoor}$) trained on the poisoned training data. For both models, we calculate their respective False Positive Rates (FPR): FPR clean represents the proportion of non-target clean nodes misclassified as the target class by the clean model, relative to the total number of non-target clean nodes. Similarly, FPR poisoned denotes the proportion misclassified by the backdoored model.

The results, as shown in Table 8, demonstrate that FPR poisoned remains very close to FPR clean across all four datasets and both model architectures. These subtle differences indicate that our backdoor attack rarely triggers abnormal misclassification behavior on clean, non-targeted nodes. The model maintains error patterns similar to the clean model, demonstrating strong stealthiness.

Table 8
Experimental results of FPR.

Dataset	GCN		GraphSage	
	FPR_clean	FPR_poisoned	FPR_clean	FPR_poisoned
Cora	0.0319	0.0540	0.0310	0.0575
Pubmed	0.0476	0.0453	0.0430	0.0469
Flickr	0.0124	0.0154	0.0156	0.0153
OGB-arxiv	0.0003	0.0010	0.0001	0.0008

4.3. Parameter analysis (RQ2)

To address Research Question 2, we conduct a series of experiments on Cora and Pubmed datasets to investigate the impact of varying poisoning rates, trigger sizes and H on the performance of backdoor attacks on Cora and Pubmed datasets.

4.3.1. Impact of poisoning rates

To investigate the influence of the poisoning rate on the ASR of backdoor attacks, we vary the poisoning rates to 1%, 2%, 5%, 10%, and 20%, without implementing any defense measures. Fig. 7 presents the experimental results, and the following observations can be made:

- **Increasing ASR with Poisoning Rate**: The ASR shows a positive correlation with the poisoning rate. On both datasets, we observe that as the poisoning rate increases, the ASR also rises, which aligns with our intuition and expectations. A higher poisoning rate strengthens the connection between the triggers and the target class, leading to a higher ASR.
- **Consistently High ASR**: The Cora dataset demonstrates a consistently high ASR across all poisoning rates, as indicated by the red line in Fig. 7. The ASR reliably surpasses 0.9, regardless of the degree of poisoning. While the Pubmed dataset exhibits slightly lower attack performance compared to Cora, the ASR still exceeds 86% in most cases. These findings highlight the resilience and efficacy of the BPBA approach, showcasing its capability to maintain a high ASR across diverse levels of poisoning.

These findings suggest that BPBA is resilient to changes in the poisoning rate and consistently achieves high ASR, making it a robust backdoor attack method. The consistently high ASR is particularly notable, as it demonstrates the attack's resilience against variations in the level of poisoning, which is crucial for maintaining the attack's effectiveness in real-world scenarios.

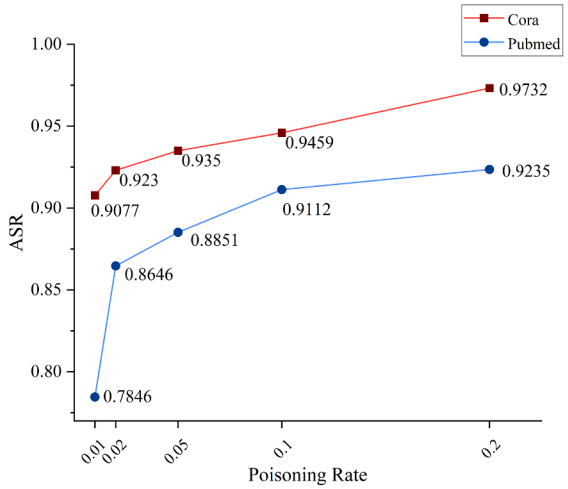


Fig. 7. Impact of poisoning rate.

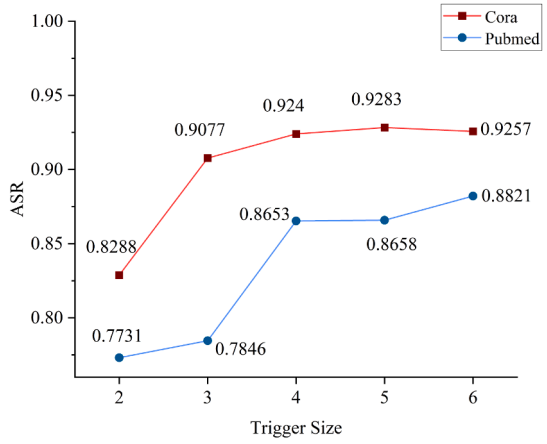


Fig. 8. The effect of trigger size on the ASR.

4.3.2. Impact of trigger size

To delve into the effect of trigger size on the ASR of backdoor attacks, we experiment with trigger sizes of 2, 3, 4, 5, and 6. Fig. 8 depicts the outcomes of these experiments, and the following observations can be drawn:

- **Increasing ASR with Trigger Size**: There is a positive correlation between the ASR and the trigger size on both datasets. As the trigger size increases, so does the ASR, which is consistent with our intuition and expectations. Larger triggers are more effective at triggering the backdoor in the model, leading to a stronger response from the neurons and thus a higher ASR.
- **Relatively High ASR Across Trigger Sizes**: The Cora dataset demonstrates a high ASR across different trigger sizes, as depicted by the red line in Fig. 8. With a trigger size of 3 or more, the ASR consistently exceeds 0.9. On the Pubmed dataset, despite relatively weaker attack performance compared to Cora, the average ASR for the five distinct trigger sizes remains over 83.42%. These results underscore the potency of BPBA in achieving substantial ASR with a range of trigger sizes.

These findings indicate that larger triggers enhance the attack's effectiveness, and BPBA remains effective even with smaller triggers, showcasing its robustness and adaptability in different scenarios. The high ASR across trigger sizes highlights the vulnerability of GNNs to such attacks and underscores the impact of BPBA's trigger design on the attack's performance.

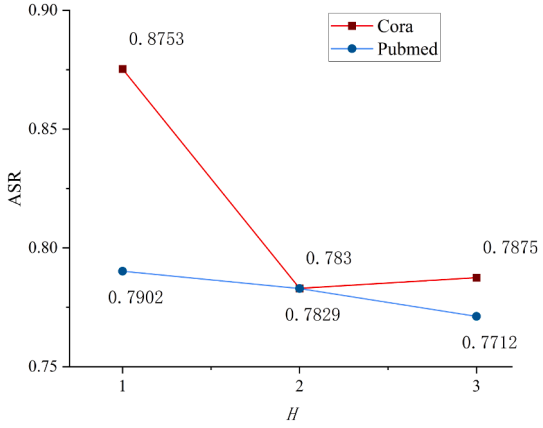


Fig. 9. The effect of H on the ASR.

4.3.3. Impact of parameter H

To investigate the impact of parameter H on the ASR of backdoor attacks, we set H to 1, 2, and 3 respectively and conducted experiments under the Prune defense setting. Fig. 10 illustrates the experimental results. It can be observed that as H increases, the ASR shows a declining trend on both the Cora and Pubmed datasets. This finding aligns with our intuition: the greater the similarity to the poisoned nodes, the better the stealthiness of their features when used as trigger node features, leading to more ideal attack outcomes. Conversely, reduced similarity results in poorer attack performance.

4.4. Ablation analysis (RQ3)

To address Research Question 3, we conduct ablation studies to assess the individual contributions of the poisoned node selection strategy and the trigger builder to the overall attack performance of BPBA.

To demonstrate the effectiveness of the poisoned node selection strategy, we develop a variant of BPBA, denoted as BPBA-without-NS, which excludes this strategy. In this variant, poisoned nodes are selected randomly in the initial stage. To evaluate the effectiveness of the trigger builder, we create another variant, BPBA-without-TB, which removes the trigger builder from BPBA. This variant employs a fixed trigger shape and size of 3, randomly generates node features for the trigger, applies the prune defense method, and conducts experiments on the Cora and Flickr datasets.

We compare the results of these two variants with the complete BPBA to illustrate the impact of each module. Fig. 9 presents the outcomes of the experiment, leading to the following observations:

- **Poisoned Node Selection Strategy**: The ASR of BPBA is higher than that of BPBA-without-NS. This is due to the poisoned node selection strategy's ability to accurately identify poisoned nodes from each non-target class, resulting in a higher ASR compared to randomly selected nodes.
- **Trigger Builder**: The ASR of BPBA is also higher than that of BPBA-without-TB. This indicates that the trigger builder is effective in constructing adaptive and stealthy triggers, thereby enhancing BPBA's attack robustness against defense mechanisms.

These results underscore the importance of both the poisoned node selection strategy and the trigger builder in the overall performance of BPBA. The strategy ensures that the attack is targeted and effective, while the trigger builder ensures that the attack is covert and resilient to detection.

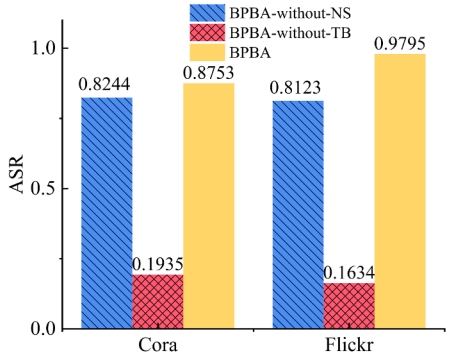


Fig. 10. Ablation studies on Cora and Flickr.

5. Related work

This section reviews GIAs and backdoor attacks, providing definitions and key works in each area. It also distinguishes between these two attack methods.

5.1. Graph injection attacks

GIAs represent an evolution of Graph Adversarial Attacks (GAAs). In GAAs, attackers aim to degrade the model's performance by altering specific edges or features within the input graph. In contrast, GIAs involve inserting new nodes into the graph, which can lead to performance degradation and misclassification without modifying existing elements. Sun et al. [28] propose a reinforcement learning approach for GIAs. Their method uses a hierarchical Q-learning network to manipulate the labels of adversarial nodes and their connections within the graph. They design a reward function to guide the learning agent in reducing the node classification performance of GNNs. Wang et al.[29]. develop a scalable framework called the Approximate Fast Gradient Sign Method (AFGSM) for GIAs. This method addresses scenarios where attackers can only inject new adversarial nodes into the graph without altering the original structure. Their approximation strategy linearizes the attacked model and derives an approximate closed-form solution with reduced time complexity. Zou et al. [24] analyze the topological vulnerability of GNNs under GIAs and introduce the Topological Defective Graph Injection Attack (TDGIA). They use a topological defective edge selection strategy to choose original nodes for connection with adversarial nodes and design a smooth feature optimization objective to generate features for these nodes.

Overall, GIAs are recognized as more sophisticated than GAAs and are inherently more stealthy. Similar to GIAs, backdoor attacks also involve the insertion of additional nodes into the graph, but their purpose differs. GIAs are designed to significantly reduce the model's overall performance by introducing new nodes, whereas backdoor attacks aim to activate covert behaviors under specific conditions to influence and control the model's behavior. This distinction highlights the divergent goals and strategies employed by these attack methods.

5.2. Backdoor attacks

Backdoor attacks aim to implant a backdoor into a model during the training phase, which can be activated during the inference phase, altering the model's behavior in a predetermined way. These attacks can be categorized into two domains: image and graph.

Image Domain Backdoor Attacks: Gu et al. [30] introduce BadNets, a classic backdoor attack method, which involves inserting specific patterns (backdoors) into a subset of the training data and altering their labels to the target label. Chen et al. [31] perform backdoor attacks on facial recognition systems by incorporating triggers (e.g., glasses) into training images. Duan et al. [32] explore backdoor attacks triggered by specific conditions rather than predefined triggers. They jointly optimize the loss function of a compression operator and the target model, using JPEG compression as the trigger condition.

Graph Domain Backdoor Attacks: Backdoor attacks in the graph domain can be divided into graph-level and node-level attacks. (1) **Graph-level Backdoor Attacks**: Zhang et al. [16] highlight the susceptibility of graphs to backdoor attacks. Xi et al. [23] propose GTA, which defines the trigger as a specific subgraph, including both topology and descriptive features. GTA dynamically adapts the trigger to individual graphs, optimizing attack effectiveness and stealthiness. It can also perform attacks without needing knowledge of the downstream model or fine-tuning strategy. Zhang et al. [33] systematically study the vulnerability of GNNs to clean-label backdoor attacks, proposing GCBA with poisoning, crafting, and natural backdoor attacks, targeting various stages of GNN training. (2) **Node-level Backdoor Attacks**: Dai et al. [15] investigate undetectable graph backdoor attacks under a limited attack budget. They selectively poison nodes during the poisoning phase to inject triggers and assign target class labels, using an adaptive trigger generator to create stealthy triggers. Yang et al. [34] propose PerCBA, a persistent clean-label backdoor attack scheme for semi-supervised node classification tasks. PerCBA uses a trigger-agnostic feature perturbation generator with adjustable budgets to create different perturbations for target and non-target classes, applying these perturbations to a small number of unmarked nodes.Zhang et al. [25] introduce DPGBA, a stealthy graph backdoor attack that leverages adversarial learning and an OOD detector to generate in-distribution triggers, while employing a memory enhancement module to ensure high ASR against graph neural networks. Ding et al. [26] propose SPEAR, a stealthy graph backdoor attack that preserves the original graph structure and perturbs only node attributes using feature importance selection and neighborhood awareness to achieve high attack success while evading defenses. Wang et al. [35] propose MLGB, the first one-to-many graph backdoor attack method, which generates label-specific triggers enforced by loss differentiation and employs innovative node selection to achieve effective and robust attacks across various GNN architectures. Xia et al. [36] propose CGBA, a clean-label graph backdoor attack that perturbs features of uncertain target-class nodes without altering labels, using original features as triggers to achieve high attack success with low poisoning rates.

Overall, backdoor attacks represent a sophisticated and novel method of attack. They offer enhanced stealth and attack effectiveness compared to traditional methods, with relatively less implementation difficulty. Furthermore, backdoor attacks provide a degree of control over the target model's behavior, which is not achievable with GAAs and GIAs. This area of research holds significant promise and warrants further exploration. In addition to the above attack methods, there are also some attacks against network security, for example Jia et al. [37] introduces ACAM, a framework for detecting multi-step cyber attacks in smart cities. It uses the MDATA model for spatio-temporal reasoning and a modular pipeline to reduce false alarms, correlate attacks, and improve detection performance.

6. Conclusion

In this paper, we empirically highlight substantial variations in ASRs when targeting distinct classes within imbalanced datasets, and we argue that creating covert triggers presents a significant challenge. To overcome these obstacles, we propose BPBA, a novel backdoor attack method designed for node classification tasks involving imbalanced datasets. BPBA is built upon a distinctive strategy for selecting poisoned nodes, which carefully selects a suitable number of nodes from each non-target class. Moreover, we introduce a trigger builder that adaptively produces more stealthy triggers, thereby improving the resilience of backdoor attacks against defensive strategies. Through extensive experimentation on multi-scale and imbalanced datasets, we convincingly

show that BPBA can launch effective, balanced, and stealthy backdoor attacks.

Our findings contribute to the understanding of backdoor attacks in the context of imbalanced datasets and provide a practical approach to designing more effective and stealthy backdoor attacks. While BPBA effectively narrows the gap in ASR between different classes, there is still room for improvement. Additionally, exploring backdoor defense methods represents a promising direction for future research.

CRediT authorship contribution statement

Haibin Tong: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Huifang Ma:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition; **Hui Shen:** Writing – review & editing, Resources, Data curation; **Zhixin Li:** Writing – review & editing, Funding acquisition; **Liang Chang:** Writing – review & editing, Funding acquisition.

Data availability

All data are public datasets.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This study is supported by National Natural Science Foundation of China (Nos. 62567007, U22A2099 and 62441701), Gansu Province Key Foundation Project (No. 24JRRA123), and the Industrial Support Project of Gansu Colleges (No. 2022CYZC11).

References

[1] D. Gamba, Y. Yu, Y. Yuan, G. Schoenebeck, D.M. Romero, Exit ripple effects: understanding the disruption of socialization networks following employee departures, in: Proceedings of the ACM on Web Conference 2024, ACM, 2024 https://doi.org/10.1145/3589334.3645634

[2] Y. Jiang, H. Ma, X. Zhang, Z. Li, L. Chang, Incorporating metapath interaction on heterogeneous information network for social recommendation, Front. Computer Sci. 18 (1) (2024) 181302.

[3] X. Zhong, H. Liang, On the scale-free property of citation networks: an empirical study, in: Companion Proceedings of the ACM on Web Conference 2024, Springer, 1 (2024), p. 181302.

[4] C. Yang, J. Han, Revisiting citation prediction with cluster-aware text-enhanced heterogeneous graph neural networks, in: 2023 IEEE 39th International Conference on Data Engineering (ICDE), IEEE, 2023, pp. 682–695.

[5] R. Zhang, H. Ma, Q. Li, Y. Wang, Z. Li, Dual-view self-supervised co-training for knowledge graph recommendation, in: International Conference on Database Systems for Advanced Applications, Springer, 2023, pp. 113–128.

[6] H. Li, G. Appleby, C.D. Brumar, R. Chang, A. Suh, Knowledge graphs in practice: characterizing their users, challenges, and visualization opportunities, IEEE Trans. Vis. Comput. Graph. 30 (1) (2023) 584–594.

[7] Liu Z., Li J., Huang Y., Cui N., Pei L.. Knowledge-based natural answer generation via effective graph learning. Knowl. Based Syst. 316 (2025) 113288.

[8] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017) 1–42.

[9] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2016.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations, 2018.

[11] Y. Chen, H. Yang, Y. Zhang, K. Ma, T. Liu, B. Han, J. Cheng, Understanding and improving graph injection attack by promoting unnoticeability, (2022) arXiv:2202.08057.

[12] Tao S., Cao Q., Shen H., Wu Y., Hou L., Sun F., Cheng X.. Adversarial camouflage for node injection attack on graphs. Inf. Sci. 649 (2023) 119611.

[13] Q. Li, H. Ma, W. Jin, Y. Ji, Z. Li, Multi-interest network with simple diffusion for multi-behavior sequential recommendation, in: Proceedings of the 2024 SIAM International Conference on Data Mining (SDM), SIAM, 2024, pp. 734–742.

[14] Y. Zhu, L. Tong, G. Li, X. Luo, K. Zhou, FocusedCleaner: sanitizing poisoned graphs for robust GNN-based node classification, IEEE Trans. Knowl. Data Eng. 36 (6) (2023) 2476–2489.

[15] E. Dai, M. Lin, X. Zhang, S. Wang, Unnoticeable backdoor attacks on graph neural networks, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 2263–2273.

[16] Z. Zhang, J. Jia, B. Wang, N.Z. Gong, Backdoor attacks to graph neural networks, in: Proceedings of the 26th ACM Symposium on Access Control Models and Technologies, 2021, pp. 15–26.

[17] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, Y. Kalantidis, Decoupling representation and classifier for long-tailed recognition, (2019) arXiv:1910.09217.

[18] W. Webber, A. Moffat, J. Zobel, A similarity measure for indefinite rankings, ACM Trans. Inf. Syst. (TOIS) 28 (4) (2010) 1–38.

[19] J. Song, J. Park, E. Yang, TAM: topology-aware margin loss for class-imbalanced node classification, in: International Conference on Machine Learning, PMLR, 2022, pp. 20369–20383.

[20] S. Prithviraj, G. Namata, M. Bilgic, L. Getoor, B. Galligher, T. Eliassi-Rad, Collective classification in network data, AI Mag. 29 (3) (2008) 93.

[21] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, Graphsaint: graph sampling based inductive learning method, (2019) arXiv:1907.04931.

[22] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open graph benchmark: datasets for machine learning on graphs, Adv. Neural Inf. Process. Syst. 33 (2020) 22118–22133.

[23] Z. Xi, R. Pang, S. Ji, T. Wang, Graph backdoor, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 1523–1540.

[24] X. Zou, Q. Zheng, Y. Dong, X. Guan, E. Kharlamov, J. Lu, J. Tang, Tdgia: effective injection attacks on graph neural networks, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 2461–2471.

[25] Zhang Z., Lin M., Dai E., Wang S.. Rethinking graph backdoor attacks: a distribution-preserving perspective. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024, p. 4386–4397.

[26] Ding Y., Liu Y., Ji Y., Wen W., He Q., Ao X.. Spear: a structure-preserving manipulation method for graph backdoor attacks. In: Proceedings of the ACM on Web Conference 2025. 2025, p. 1237–1247.

[27] D. Zhu, Z. Zhang, P. Cui, W. Zhu, Robust graph convolutional networks against adversarial attacks, in: Proceedings of the 25Th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 1399–1407.

[28] Y. Sun, S. Wang, X. Tang, T.-Y. Hsieh, V. Honavar, Adversarial attacks on graph neural networks via node injections: a hierarchical reinforcement learning approach, in: Proceedings of the Web Conference 2020, 2020, pp. 673–683.

[29] J. Wang, M. Luo, F. Suya, J. Li, Z. Yang, Q. Zheng, Scalable attack on graph data by injecting vicious nodes, Data Min. Knowl. Discov. 34 (2020) 1363–1389.

[30] T. Gu, B. Dolan-Gavitt, S. Garg, BadNets: Identifying vulnerabilities in the machine learning model supply chain, (2017) arXiv:1708.06733.

[31] X. Chen, C. Liu, B. Li, K. Lu, D. Song, Targeted backdoor attacks on deep learning systems using data poisoning, (2017) arXiv:1712.05526.

[32] Q. Duan, Z. Hua, Q. Liao, Y. Zhang, L.Y. Zhang, Conditional backdoor attack via JPEG compression, in: Proceedings of the AAAI Conference on Artificial Intelligence, 38, 2024, pp. 19823–19831.

[33] H. Zhang, J. Chen, L. Lin, J. Jia, D. Wu, Graph contrastive backdoor attacks, in: International Conference on Machine Learning, PMLR, 2023, pp. 40888–40910.

[34] X. Yang, G. Li, C. Zhang, M. Han, W. Yang, PerCBA: persistent clean-label backdoor attacks on semi-supervised graph node classification (2023).

[35] Wang K., Deng H., Xu Y., Liu Z., Fang Y.. Multi-target label backdoor attacks on graph neural networks. Pattern Recognit. 152 (2024) 110449.

[36] Xia H., Zhao X., Zhang R., Xu S., Wang L.. Clean-label graph backdoor attack in the node classification task. In: Proceedings of the AAAI Conference on Artificial Intelligence; vol. 39. 2025, p. 21626–21634.

[37] Jia Y., Gu Z., Du L., Long Y., Wang Y., Li J., Zhang Y.. Artificial intelligence enabled cyber security defense for smart cities: a novel attack detection framework based on the mdata model. Knowl. Based Syst. 276 (2023) 110781.